# CS 260: Data Structures

Abstract Data Types

# Assertions

- Assertions are statements used to test assumptions made by programmers
- If assert is not met, then program aborts and error message is displayed
- Use assertions to enforce pre-conditions
- Requires assert.h
  - #include <assert.h>

- Example
  - //precondition - x is greater than 0
  - Assert(x > 0);

# Assert Example

- Create a function called countdown
- Precondition: argument is greater than 0

# Abstraction

- Removing excessive detail
- Consistent theme in computer science
- Create simplified models or representations of complex systems

# Data Structures and ADTs

- Abstract Data Type
  - A mathematical description of a model which is defined by its operations
  - Focuses on what operations can be performed rather than how they are executed
  - Theoretical
  - ADTs are abstract, language is unimportant
  - What can it do as opposed to how it is done
- Data Structure
  - Data structures refer to the way data is organized, stored, and accessed in a computer
  - Concrete implementation
  - The implementation of an ADT is a data structure
  - We will use C to implement data structures

# Abstract Data Types

- Often described by real-world examples
  - Bag, Stack, queue, tree, etc



# Abstract Data Types

- Levels of abstraction
- ADT
  - What is it?
- Interface
  - What are the associated operations?
- Implementation
  - How is it implemented?

# General ADT Examples

- Bag
  - Simple unordered collection of items
- Stack, queue, deque
  - Ordered by insertion order
- Trees
  - Binary trees, heaps, etc
  - Ordered in some way

# What is a Bag?

- A bag is an ADT which holds a collection of items
  - Items are unordered
  - There can be duplicate items
  - Items are typically homogenous (same data type)
  - Capacity may or may not be finite
  - Unordered

  - Complexity varies based on implementation
    - Array v linked list
      - Insert implementation is the primary operation



# Bag Interface

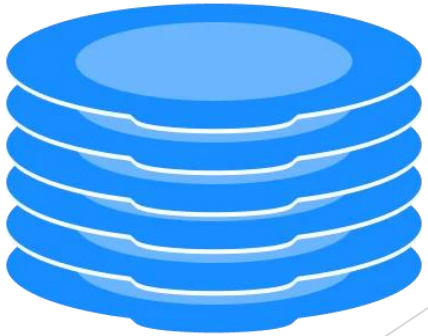
- Define the operations
  - Complexity of operations is generally known
- What might we want to do with a bag?
  - Init
  - Destroy
  - Add
  - Display
  - Contains
  - Remove
- Auxiliary functions
  - getSize
  - isEmpty
  - isFull
  - etc

# Bag Implementation

```
/*Int array is used to store elements*/
Struct bag{
        int size;/*number of elements*/
        int cap;/*capacity*/
        int b*;/*Storage of elements*/
};
Void initBag (struct bag *b, int cap);
Void addBag (struct bag *b, int item);
Void displayBag (struct bag *b);
Int removeBag (struct bag *b, int item);
Int containsBag (struct bag *b, int item);
```

# What is a Stack?

- A container of objects that are inserted and removed in the order received
  - Last-in first-out (LIFO)
  - There can be duplicate items
  - Items are typically homogenous (same data type)
  - Capacity may or may not be finite
  - Ordered but not sorted
  - Complexity varies based on implementation
    - Array v linked list
      - Insert implementation is the primary operation



# Stack Interface

- Define the operations
  - Complexity of operations is generally known
- Basic operations
  - Init
  - Destroy
  - Push
  - Pop
  - Peek
- Auxiliary functions
  - isEmpty
  - getSize
  - isEmpty
  - isFull
  - etc

# What is a Queue?

- A container of objects where the last item inserted is the first to be removed
  - First-in first-out (FIFO)
  - There can be duplicate items
  - Items are typically homogenous (same data type)
  - Capacity may or may not be finite
  - Ordered but not sorted
  - Complexity varies based on implementation
    - Array v linked list
      - Insert implementation is the primary operation



# What Would We Want to do With a Queue?

- Basic operations
  - Init
  - Destroy
  - Push (enqueue)
  - Pop (dequeue)
  - Peek
- Auxiliary functions
  - isEmpty
  - getSize
  - isEmpty
  - isFull
  - etc

# What is a Deque?

- Double ended queue
- Elements can be added to the head or tail
  - front / back can be confusing (lack of consistency)



# What Would We Want to do With a Deque?

- Basic operations
  - Init
  - Destroy
  - PushFront
  - PopFront
  - PushBack
  - PopBack
  - PeekFront
  - PeekBack
- Auxiliary functions
  - isEmpty
  - getSize
  - isEmpty
  - isFull
  - etc

# Dynamic Arrays

- An array that can resize itself
- Normal arrays are of fixed size
- Also called vectors or other things
- In C we don’t have dynamic arrays
  - But we can build them

# What Would We Want to do With a Dynamic Array?

- Basic operations
  - Init
  - Insert
  - Delete
  - Search
- Auxiliary functions
  - Typical operations

# What do we Need in Our Dynamic Array Implementation?

- Capacity
- Size
- Array to store the data

# Insert Into a Dynamic Array

- Add an element to an array is more complicated than a stack, queue or bag
- It might go somewhere other than the front or back
- Several cases
  - Add to the end
  - Add before the end
  - Add to full array

# Growing a Dynamic Array

- This is necessary when inserting into a full array
- How much do we want to grow the array?
- We need to copy the array
  - In C we can use realloc
  - Copy is handled for us but it still happens
    - Costly operation

# Delete From a Dynamic Array

- Remove from end
- Remove from anything other than the end
- Might want to shrink
  - How much do we want to shrink the array?
  - Why would we need to do this?

# Dynamic Array Implementation

```
/*Int array is used to store elements*/
Struct DynArr{
        int size;/*number of elements*/
        int cap;/*capacity*/
        int d*;/*Storage of elements*/
};
Void initDynArr (struct DynArr *d, int cap);
Void addDynArr (struct DynArr *d, int item);
Int removeDynArr (struct DynArr *d, int item);
Void displayDynArr (struct DynArr *d);
Int containsDynArr (struct DynArr *d, int item);
Etc.
```

# Stack Implementation Using a Dynamic Array

- Where should we push?
  - Why?
- Where should pop?
  - Why?
- Where should we peek?
  - Why?

# Lab

- Build a dynamic array using a skeleton that I have provided
  - This should familiarize you with what you need to understand about skeleton code